Implementing Robust JPEG Steganography Using Wavelet Domain SVD and Adaptive QIM

CS 5XXX Steganography

Team 22
Winkelmann17
Awesome Partner

May 26th – August 4th, 2026

Introduction 3
The Goal 3
Key Differences of Our Implementation 3
Algorithm Design and Testing 4
Packages and Script Details 4
Embedding Diagram 7
Extracting Diagram 9
Test Images and Files 10
Analysis and Comparison 12
General Findings of our Implementation 14
Capacity 16
Perceptibility and Detection 17
Robustness 18
Review of Comparison to F5 18
Challenges and Future Work 20
Conclusion 21
Bibliography 22
Resources 22

Introduction

There are three core aspects of Steganography: security, capacity, and robustness. Ideally, each of these core goals is maximized, but oftentimes tradeoffs must be made. Robustness, the ability to maintain data integrity through manipulation, often comes at the expense of capacity, the amount of data that can be hidden within a cover image. JPEGs are one of the most common image formats that are seen today. As such, they are great candidates for steganography due to their wide use. JPEG images undergo an extensive compression algorithm that utilizes discrete cosine transforms (DCT), run-length compression, and Huffman compression. Because of this, JPEG steganography is necessarily mathematically complex. This complexity can be an aid in efforts to thwart detection, which is one of the principal goals of the following chosen algorithm.

The Goal

Jing Liu, Xiaofeng Song, Guopeng Li, and Kun Han propose a robust method of JPEG steganography that utilizes wavelet domain singular value decomposition (SVD) and quantization index modulation (QIM). Our goal is to implement this steganographic technique through Python code and analyze its effectiveness against its stated goals of robustness and thwarting detection. We will analyze Liu et al.'s claims through these analyses and then compare the performance of our implementation against a Python implementation of the F5 JPEG steganography algorithm proposed by Westfeld, which embeds data into quantized non-zero AC DCT coefficients using password-based pseudorandom coefficient permutation, matrix encoding, and shrinkage handling. This comparison will allow us to determine whether the wavelet domain SVD and adaptive QIM approach provides measurable improvements over an established JPEG steganography technique.

Key Differences of Our Implementation

For our Python implementation, there are some key differences or expansions of the algorithm. The biggest difference is that, due to the limited time available during the semester, we did not focus on implementing the syndrome-trellis code (STC) operations found within the proposed Wavelet-SVD algorithm. Another difference is that Liu et al.'s algorithm proposal lacks some details such as the specific wavelet transformation. This absence was absorbed by our implementation by offering the ability to choose from a variety of transforms on the commandline. Some other commandline options were included, such as being able to choose a block size other than 4 and changing the Ti values by changing specifying minimum and maximum quantization values. We also extended the functionality to include performing the algorithm with no Ti key file being provided at extraction. This *does* lower the extraction rate substantially, but it is possible to gather data without the keyfile, which is not something proposed by Liu et al.

The biggest difference is that of extraction performance; our implementation does not achieve a 100% extraction rate. There are a few suspected reasons for this, the second of which is also a key difference of our implementation compared to Liu et al.'s.

1. Our algorithm opens the JPEG in Pillow, Python's image processing library, modifies data, and then saves a *new* JPEG image. This introduces the lossy compression of creating a brand new stego JPEG.
2. STCs and the weighted block embedding were not implemented due to time constraints. This could've influenced choosing weaker blocks that do not hold their bit parity through the transforms and recompression when the new JPEG is written.
3. The specific experiments and evaluation parameters of the paper are not well-outlined or defined. The images chosen by the authors are random images from the BOSSbase1.01 database, but we did not use that source (Liu et al., 2). The images we used were chosen for entertainment/fun and the large sizes required to test the embedding of larger messages/files. The paper also does not describe the type of messages they are embedding, though we doubt that really has a substantial effect.

Algorithm Design and Testing

For our implementation, Python was chosen for ease of use and readability. Python has extensive, easy-to-spin-up library support that vastly improved the development experience of our implementation. Python was also seen as the most portable language to use, being able to run on both Windows and Linux without significant overhead or setup. With this in mind, we will now discuss the specific packages that are used by our implementation and why they were chosen, as well as the updated block diagrams for both embedding and extraction portions of the algorithm.

Packages and Script Details

As previously stated, the implementation of this JPEG steganography algorithm will be done using Python. Before getting into the functional block diagrams, it is important to note what packages will need to be installed. The following packages and their purpose have been identified with their specific use cases.

- **PyWavelets**
  - Open-source package for implementing wavelet transforms in Python
  - Will specifically be used for implementing the forward and inverse stationary wavelet transforms (SWT)
- **NumPy**
  - Open-source package for handling more mathematically complex operations, especially powerful with arrays

    - NumPy has built in SVD functions, standard deviation, and mean functions that will be used for block operations
    - NumPy is heavily used by other packages (and thus is required) and has its own array objects that have more functions and methods specific for matrix math
- **Pillow**
    - A modern, maintained image processing library for Python
    - Has extensive JPEG support and can split relevant data segments automatically, reducing overhead of sorting through JPG hex markers and data
- **Argparse**
    - Python's in-built module for handling command-line inputs
    - This will be the chief way the user can operate our program and choose options such as embedding, extracting, and the output file name
- **Pickle**
    - Python's in-built data serialization module
    - This will be the chief way the Ti key file will be saved to disk during the embedding process. No specifc method of key generation was specified by Liu et al., so Pickle was chosen for its simplicity of reading/writing arrays to disk.

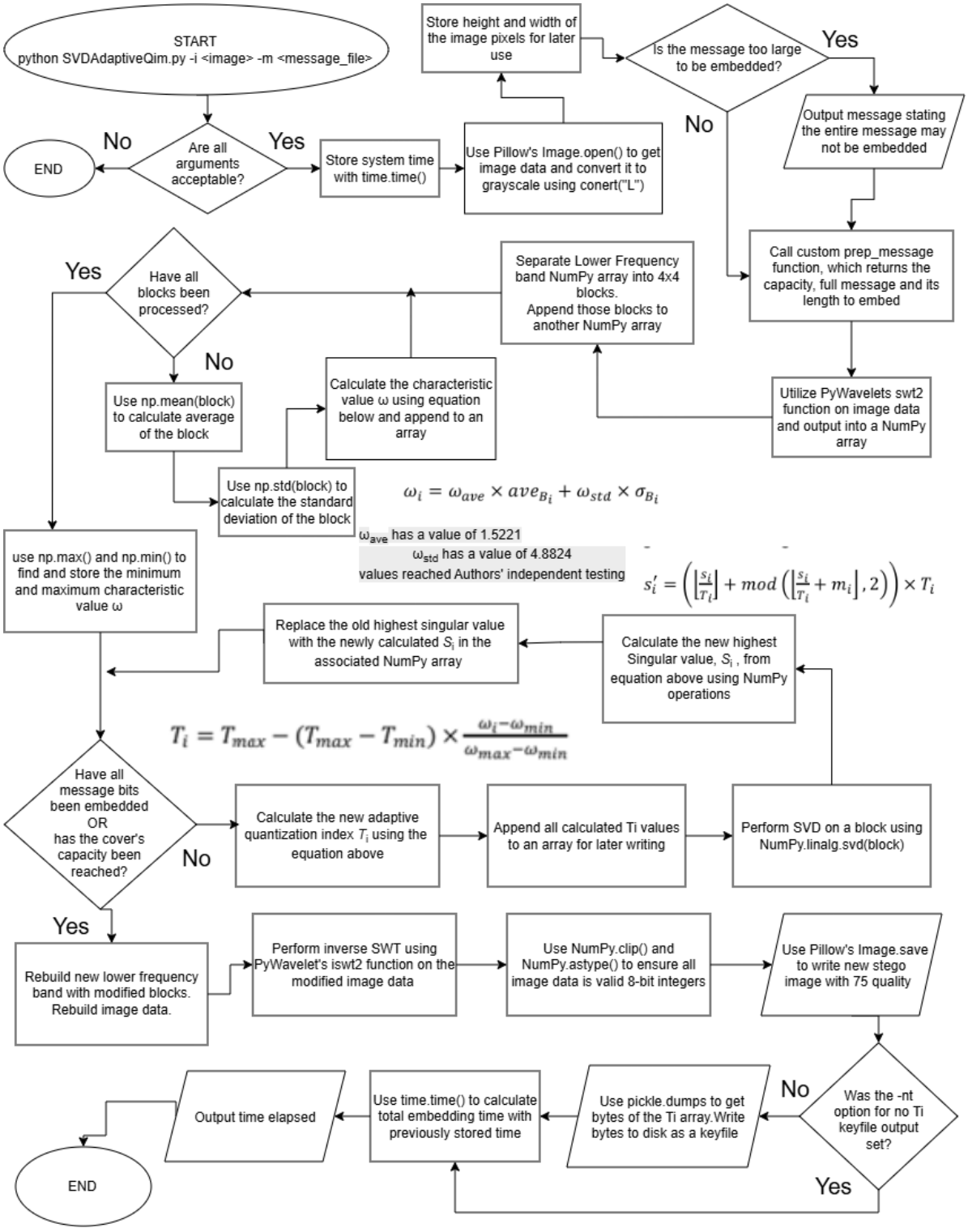
The implementation of this algorithm will be done through a Python 3 script. This script will require the user to supply an image and a message for embedding or a stego-image and the appropriate option for extraction. This will all be done through the command line via the *Argparse* Python library with specific flags being set on the command line: *–i* for the image path, *–m* for the path to the message data, and *–extract* to extract from the image. Help messages and error handling will be built into the program to ensure ease of use. There is a total of seven other options included in the program:

- -Tk : Used for specifying the path to the Ti key generated at embedding during extraction. This tells the extraction algorithm to use the specific Ti's generated at embedding.
- -b : Used to specify a block size other than 4. Completely optional, defaults to 4 if not given.
- -w : Used to specify the specific wavelet to use for the stationary wavelet transforms. The options available are in the help message. Defaults to "haar."
- -Tmin : Used to specify the minimum quantization value, Tmin, if the default of 20 is not desirable.
- -Tmax : Used to specify the maximum quantization value, Tmax, if the default of 70 is not desirable.
- -nt : Used with embedding to forego generation of a Ti key file.

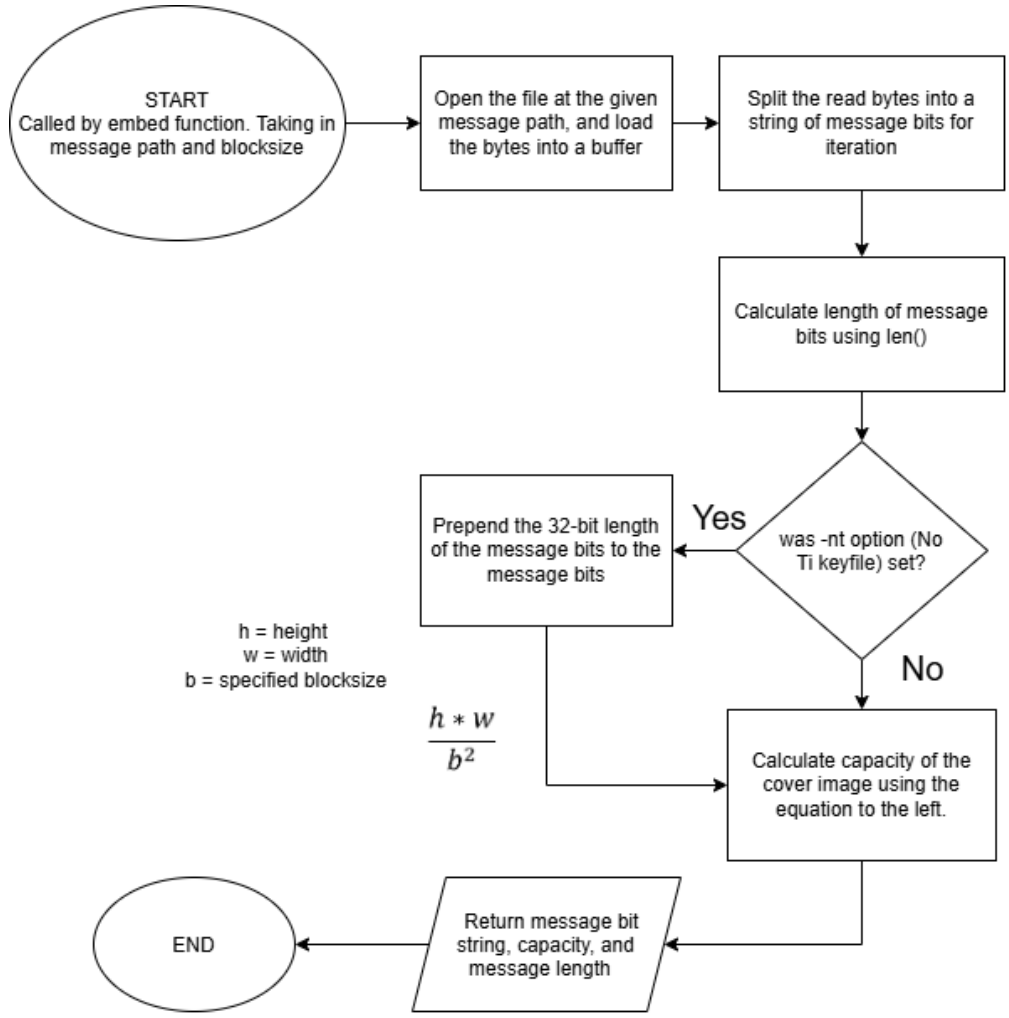
- -o : Used specify the output path that the stego images or extracted data will be written to.

Should a message be too large to be completely embedded within the JPEG, a message will be displayed to the user informing them that only a portion of the message will be embedded within the provided image. As execution runs, the user will view descriptive progress update messages that explain what parts of the algorithm are running to gauge completion time. In addition, a time delta will also be calculated to display to the user how long each embedding or extraction action takes. With this background complete, the specific implementation strategy of the embedding and extracting algorithms is shown in the following block diagrams.

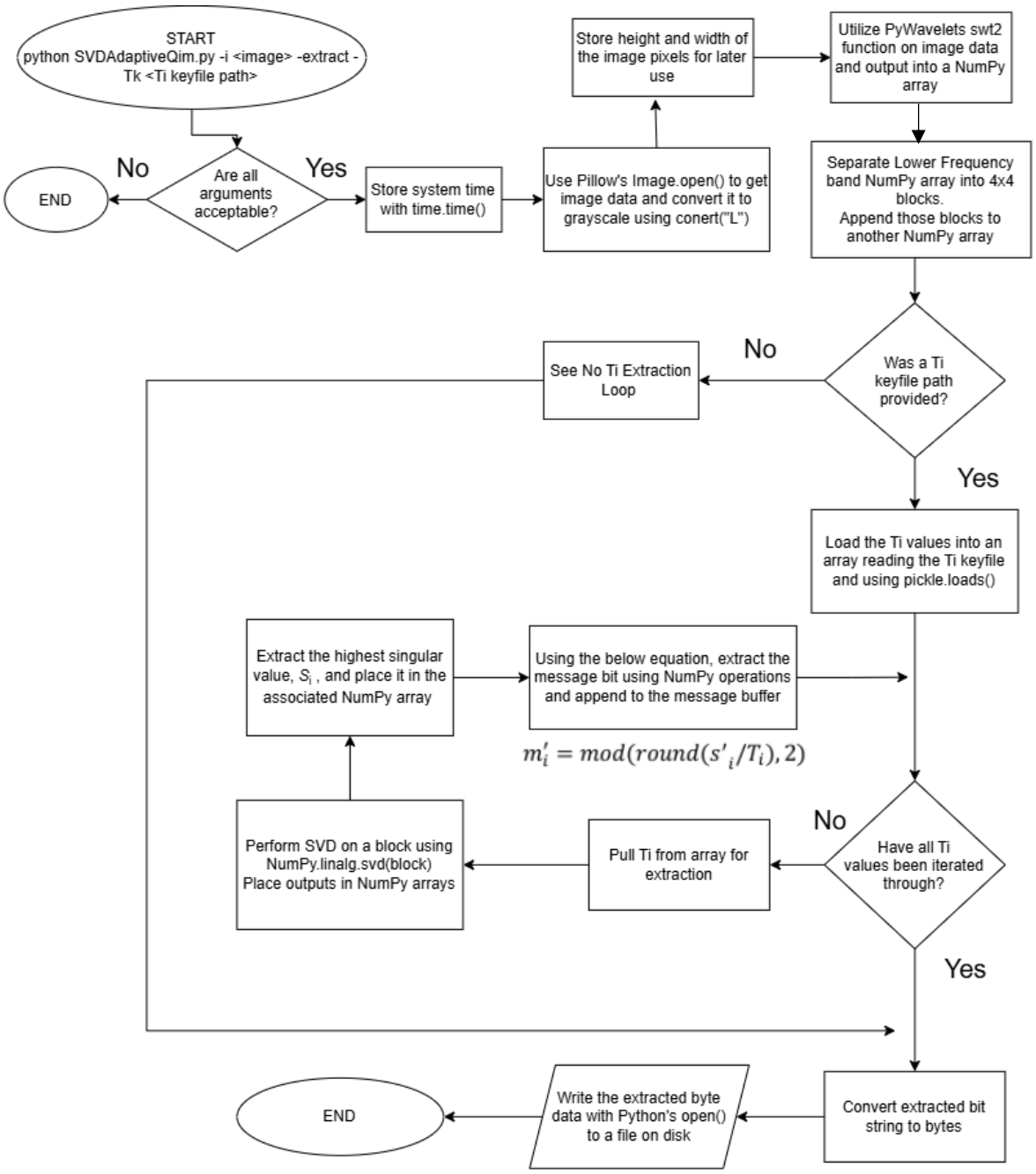
Embedding Diagram



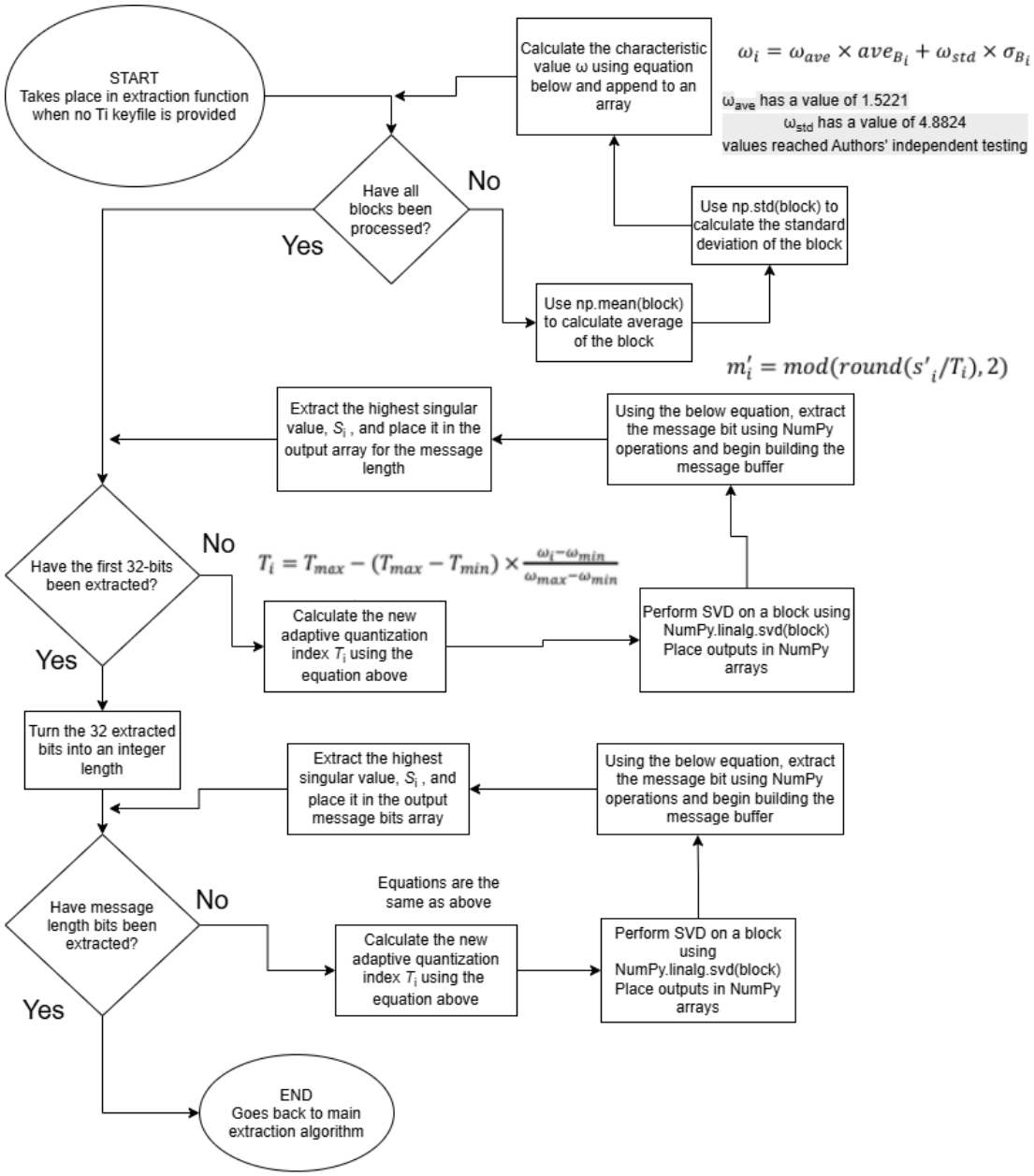
prep_message function



Extracting Diagram



No Ti Extraction Loop



Test Images and Files

To be as accurate as possible to the testing suite outlined by Liu et al., it was important to build a small library of JPEGs with 75 quality. 19 images were tested in a variety of ways. These images range from natural images, cartoon images, and rendered art. Some images were chosen specifically because they were deemed fun. All images were of varying sizes to demonstrate the embedding and extracting functionality, as well as to test capacity, and the overall effects of the algorithm on different sized images. Different styles of images may have drastic effects on the

perceptibility or efficiency of the algorithm. This algorithm is designed specifically to work on grayscale images (Liu et al, 2), but should a color image be given as input, it will be converted to grayscale automatically. Within this limitation of grayscale images, as stated prior, cartoon images, natural images, and what we call "rendered" images were utilized. Cartoon images, being ones that have very little color variation between pixels, natural images being ones that are emblematic of the real world, and rendered images being ones that are created via computer generation or design. Examples of all these images are shown below. ***NOTE:*** *None of these images are ours; we do not seek to claim these as our own.*



Fig 1: Color to Grayscale



Fig 2: Rendered image



Fig 3: Cartoon image



Fig 4: Natural image

The largest of the tested images was 4000x6000 pixels, and the smallest was 256x256 pixels. The test images were all chosen to have their pixel dimensions cleanly divisible by a block size of 4. Some "bad" images were also used to ensure our program could error check and ensure it would only use images in which the block size would evenly fit, though this restriction would most likely not be carried forward in future iterations. These grayscale images are stegged according to the algorithm specified in the above sections and are output to new stego JPEGs of 75 quality. These JPEGs will then be converted to lower quality levels to test their robustness.

This was the exact goal of the original implementation of the algorithm, to ensure that data hidden in JPEGs via steganography can persist through further compression on social media or image sharing sites (Liu et al, 1). Since further JPEG compression when degrading quality is lossy, it is fair to assume other channels of loss or destruction may be abated by this steganographic technique.

The specific data embedded within these test images varied from short MP3 files, small PNG icons, and text. These are provided with the program but are not detailed in this document as they are ultimately inconsequential to the execution of the program.

Analysis and Comparison

To evaluate the Wavelet Domain SVD and Adaptive QIM algorithm, we compared it against an existing Python implementation of the F5 JPEG steganography algorithm by jackfengji. F5 was selected as a baseline because it is a well-established JPEG steganography method that embeds data directly into quantized DCT coefficients.

The comparison focused on six evaluation categories: embedding capacity, imperceptibility, extraction accuracy, robustness, execution time, and detectability. The same cover images and payload files were used for both algorithms to ensure a fair comparison.

The evaluation was automated using a benchmark script. For each cover image and payload, the benchmark embedded the payload using both SVD Adaptive QIM and F5, extracted the hidden payload, compared the recovered file against the original payload, calculated Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM), recorded embedding and extraction times, and evaluated robustness by recompressing each stego image at JPEG quality levels of 90, 75, and 50 before attempting extraction.

Representative benchmark results are shown in Tables 1 and 2. Basic Extract indicates whether the original payload was successfully recovered prior to any JPEG recompression. Robustness indicates whether the payload remained recoverable after JPEG recompression at quality levels of 90, 75, and 50.

Test Case	Algorithm	Capacity	Payload	PSNR	SSIM	Basic Extract	Robustness
Cat + HelloWorld	SVD Adaptive QIM	1875 B	12 B	51.80	0.999966	No	No
Cat + HelloWorld	F5	3727 B	12 B	59.28	0.999994	Yes	No

Peacock + HelloWorld	SVD Adaptive QIM	7200 B	12 B	59.99	0.999994	Yes	Yes
Peacock + HelloWorld	F5	9264 B	12 B	65.30	0.999998	Yes	Partial

Table 1. Representative benchmark comparison between the proposed Wavelet Domain SVD and Adaptive QIM implementation and the F5 algorithm.

Note: For the robustness metric, "Yes" indicates successful payload recovery after recompression at all tested JPEG quality levels, "No" indicates complete recovery failure, and "Partial" indicates successful recovery at one or more, but not all, recompression quality levels.

Algorithm	Average Embed Time	Average Extract Time	General Observation
SVD Adaptive QIM	3.11 s	1.94 s	Faster execution but variable extraction reliability
F5	6.44 s	3.53 s	Slower execution with consistently successful extraction

Table 2. Average embedding and extraction times for the proposed implementation and F5.

The benchmark demonstrated several clear performance differences between the two algorithms. Across all benchmark cases, the F5 implementation consistently achieved higher PSNR and SSIM values than the proposed SVD Adaptive QIM implementation, indicating lower measurable image distortion while maintaining successful payload recovery. F5 also successfully recovered the embedded payload during every basic extraction test, whereas the proposed implementation exhibited extraction failures on several benchmark images and payload sizes. These results indicate that the extraction reliability of the proposed implementation was highly dependent on the selected cover image and payload.

Execution time varied with image size. For the smaller benchmark image (75Cat.jpg), both algorithms exhibited comparable embedding and extraction times. However, for the larger benchmark image (75Peacock.jpg), the proposed implementation consistently required less time for both embedding and extraction than the F5 implementation. This suggests that the computational overhead of the proposed algorithm scales more favorably for larger images under the tested conditions.

The most notable result was observed during the robustness evaluation. Using the 75Peacock.jpg image with the HelloWorld.txt payload, the proposed SVD Adaptive QIM implementation

successfully recovered the embedded payload after JPEG recompression at quality levels of 90, 75, and 50. Under the same conditions, the F5 implementation successfully recovered the payload only after recompression at quality level 75. Although this result represents a single benchmark case rather than a universal outcome, it demonstrates the compression robustness that the original Wavelet Domain SVD and Adaptive QIM algorithm was specifically designed to achieve.

A complete steganalysis evaluation was beyond the scope of this project. Consequently, detectability was not quantitatively evaluated using automated steganalysis software. However, Aletheia was identified as a suitable candidate for future evaluation because it supports JPEG steganalysis and includes detection methods for F5 and several adaptive embedding techniques. StegExpose was considered but was not selected because it is designed primarily for LSB steganography in lossless image formats such as PNG and BMP, whereas this project focuses exclusively on JPEG steganography.

The benchmark results provide an overall comparison between the proposed implementation and F5. The following sections examine the observed behavior of the proposed algorithm in greater detail and discuss the factors that influenced its capacity, perceptibility, robustness, and overall performance.

General Findings of our Implementation

When developing and testing the algorithm, there were a lot of general details about this algorithm that became clear. This section will attempt to showcase those findings in a general manner.

Overview of Program Success, Extraction Error Rates

When testing the algorithm, it is apparent that our implementation does not have a 100% recovery rate. There are bits that are not extracted correctly, and the specific reason for this is not immediately known. However, what was observed was when testing the algorithm; the embedding/extraction process was nearly 100% reversible before saving the modified image data to a *new* JPEG of 75 quality. This reversibility appears to be removed after using the Pillow library to save the stego JPEG, which involves some lossy compression. While this compression seems to cause the error rate, the general purpose behind the algorithm still stands. The data is still *generally* recoverable, though there may be many other factors affecting this. For example, hiding Tank.txt in 75Peacock.jpg has a 0.16% error rate when output to 75 quality, with the parity of only 3 bits being flipped. When extracting the message, it is clearly readable, though the exact characters are different in 3 spots due to the bit flips. When the stego JPEG is further compressed when converted to a quality of 50, the error rate becomes .64% with 12 bits being wrong out of 1,856 total bits. Again, the message is still entirely intelligible despite some errors.

This can be true with larger files as well. By hiding Puma.mp3 in 75Lake.jpg, and then extracting the MP3, the extracted MP3 is able to be listened to. This specific extraction had

1,929 errors out of 499,952 bits, or an error rate of 0.39%. This is still a low margin of error, but it does have a noticeable effect on the quality of the audio when played back. Even when the stego JPEG is converted to a quality of 50 after completion of the embedding process, the MP3 is recoverable and listenable (with much more distortion though) and has an error rate of 0.72%.

With all these figures in mind, it is clear that our algorithm implementation does indeed work and remains robust through further quality degradation (and subsequent lossy compression). Our implementation demonstrates the premier goals of Liu et al. in their work, though our extraction error rates are higher than what was reported by the authors. This is most likely due to the absence of STC encoding and distortion cost calculations. These steps are included in the author's paper, and most likely take their implementation to the near 0% extraction error rate that they boast of (Liu et al., 5). Without these, it is possible that the JPEG compression of the new stego JPEG causes more damage to the embedded data as explained previously. It must be noted that our implementation, because of these low error rates, does not do well with files that suffer under volatility.

Effect of Larger Blocksizes

While the authors used a blocksize of 4, our implementation offers the ability to have larger or smaller block sizes. These sizes change the number of values available for the embedding calculations and thus have a vastly different effect on an image when embedding. For the obvious findings here, increasing the block size significantly lowered capacity. This is expected, as capacity and block size are inversely related in this algorithm. Should the block size be larger, the capacity dramatically decreases. Using 75Jake.jpg we can see this comparison easily, both visually and technically. On the technical level, the Tank.txt file has a .38% error rate at the default block size of 4. However, when using a block size of 8, the extracted text was entirely intact with a 0% extraction error rate. On the visual level, below is the stego image at a block size of 4 and a block size of 8 which shows the difference in perceptibility and capacity at each block size.

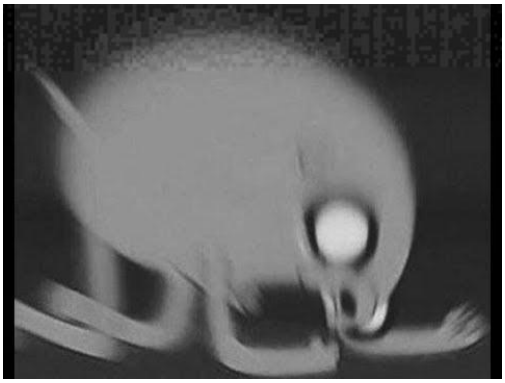


Fig 5: Jake embedded at block size 4

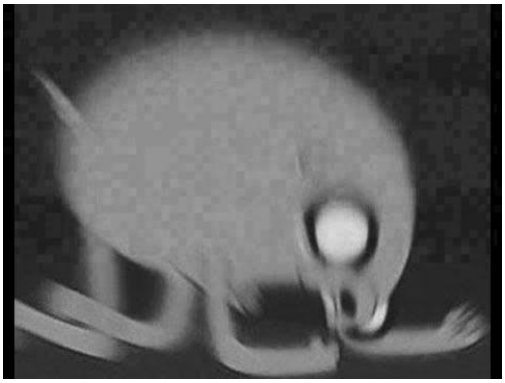


Fig 6: Jake embedded at block size 8

From experimentation, there does seem to be an upper limit to the block size, but this was not a major focus of testing, as increasing the block size severely limits capacity, and with lower capacity, true effectiveness is harder to measure. In addition, it is hard to truly pinpoint whether the block size is the source of the issues in these cases.

Effect of Larger Tmin and Tmax Values

As pointed out in the original paper, the larger the specific quantization value of a block, the more likely extraction will be correct, with adverse effects on the perceptibility and distortion of the image (Liu et al., 2). To add more functionality to our implementation, Tmin and Tmax quantization values are specifiable at the commandline. To really test and experiment with this, it is necessary to use larger payloads like the MP3 files. For testing, 75Water.jpg was embedded with George.mp3. George.mp3 is recoverable and able to be listened to but has large amounts of audio distortion. However, the audio distortion fades as the Tmin and Tmax values are increased. At the default values of Tmin=20 and Tmax=70, the bit error rate was 0.35%; at Tmin=100 and Tmax=150, the bit error rate was 0.14%; at Tmin=150 and Tmax=200, the bit error rate was 0.075%. Conversely, as the error rates and audio distortion decreased, the visible distortion in the stego image increased. This can be seen below. ***NOTE:*** *These images were rather large to accommodate the size of an MP3, so they were cropped to fit in this document.*



Figures 7, 8, 9: Increasing distortion due to increases in Tmin and Tmax values

Effects of Different Wavelets

Different wavelets did seem to have some effect on the extraction rate, though it was relatively minor. Since Liu et al. did not specify what wavelet was used in their implementation; we decided to allow the user to choose from a couple of different wavelets. While the results did vary, they were close enough to be consistent without any major variations. More could be tested and explored here in the future.

Capacity

The embedding capacity of the proposed Wavelet Domain SVD and Adaptive QIM algorithm is directly influenced by the selected block size. Because each image block stores a single embedded bit, smaller block sizes provide more embedding locations and therefore higher

capacity, while larger block sizes reduce capacity but generally improve robustness and image quality. Although the implementation supports multiple block sizes, the evaluation focused primarily on a 4×4 block size, as recommended in the original paper. This configuration provided the best balance between embedding capacity, extraction reliability, and robustness against JPEG recompression.

Experimental results showed that the proposed implementation provides a moderate embedding capacity that is sufficient for hiding small text files and messages but is lower than the capacity achieved by F5. For example, the benchmark estimated capacities of approximately 1,875 bytes for the 75Cat image and 7,200 bytes for the 75Peacock image using the 4×4 configuration. These values were consistently lower than the corresponding F5 capacities measured on the same images. This reduction in capacity is an expected consequence of embedding information within wavelet-domain singular values, where robustness is prioritized over maximizing the number of embedded bits.

Overall, the proposed implementation demonstrates a practical tradeoff between embedding capacity and robustness. While it does not match F5 in maximum payload size, the selected 4×4 configuration provides sufficient capacity for typical text-based payloads while supporting the algorithm's primary objective of maintaining resilience to JPEG recompression.

Perceptibility and Detection

The proposed Wavelet Domain SVD and Adaptive QIM algorithm generally produced stego images with minimal visible distortion, although perceptibility varied depending on the cover image, embedding parameters, and payload size. Images containing greater texture and high-frequency detail were better able to conceal embedded modifications, while smoother images exhibited more noticeable changes as the payload size increased. Quantitatively, the benchmark produced high PSNR and SSIM values across the evaluated test cases, indicating that the visual quality of the stego images remained close to the original cover images despite the embedded payload.

Several implementation parameters influenced perceptibility. Smaller block sizes increased embedding capacity but also introduced more modifications throughout the image, making visual distortion more likely. Conversely, larger block sizes reduced the number of modified regions and generally improved visual quality at the cost of reduced capacity. The choice of wavelet transforms also affected the distribution of embedded changes, although the Haar wavelet provided the most consistent results during testing and was therefore used throughout the primary evaluation. Similarly, larger payloads required modification of more image blocks, increasing the probability of detectable distortion.

No dedicated steganalysis tool was incorporated into the benchmark, so the implementation was not evaluated against statistical detection methods such as Aletheia or Stegdetect. Consequently, detectability was assessed primarily through visual inspection and image quality metrics rather

than automated steganalysis. Future work should include evaluation using established JPEG steganalysis tools to quantify resistance against statistical detection techniques and provide a more comprehensive assessment of the algorithm's security.

Robustness

Robustness was the primary focus of the proposed Wavelet Domain SVD and Adaptive QIM implementation. Unlike JPEG steganography methods that prioritize capacity or imperceptibility, this algorithm was designed to preserve the embedded payload after the stego image undergoes additional JPEG compression. This method supports this goal by embedding information into the largest singular values of wavelet-domain image blocks using Adaptive Quantization Index Modulation (Adaptive QIM), making the embedded data more resistant to the quantization effects introduced during recompression.

To evaluate robustness, stego images were recompressed at JPEG quality levels of 90, 75, and 50 before extraction was attempted. Successful payload recovery after recompression was used as the primary robustness measure. The benchmark showed that robustness depended heavily on the cover image and embedding parameters. Some test cases failed to recover the complete payload after recompression, while others, particularly 75Peacock.jpg with the HelloWorld.txt payload, successfully recovered the hidden message at all evaluated quality levels. This result supports the original paper's goal of improving payload recovery after JPEG recompression.

Compared with F5, the proposed implementation demonstrated stronger robustness in selected benchmark cases. Although F5 generally achieved higher image quality and greater embedding capacity, its extraction reliability decreased after aggressive JPEG recompression. In contrast, the Adaptive QIM implementation successfully recovered the payload in cases where F5 achieved only partial recovery or failed to recover the complete message. These results suggest that the proposed implementation captures the main design tradeoff of Wavelet Domain SVD and Adaptive QIM steganography: sacrificing some capacity and imperceptibility in exchange for improved resistance to JPEG recompression.

Review of Comparison to F5

The proposed Wavelet Domain SVD and Adaptive QIM implementation was compared against the F5 JPEG steganography algorithm using identical cover images, payloads, and benchmark procedures. Although both algorithms successfully embedded hidden information within JPEG images, their performance differed across the evaluated metrics.

In terms of embedding capacity, F5 consistently supported larger payloads than the proposed implementation. For the benchmark images, F5 achieved capacities of 3,727 bytes for the 75Cat image and 9,264 bytes for the 75Peacock image, while the proposed implementation achieved capacities of 1,875 bytes and 7,200 bytes, respectively. This difference is expected because F5 embeds information directly into quantized JPEG DCT coefficients using matrix encoding,

whereas the proposed implementation embeds one bit within each wavelet-domain image block, making its capacity dependent on the selected block size.

For imperceptibility, F5 consistently produced higher PSNR and SSIM values across every benchmark case. These results indicate that F5 introduced less measurable image distortion while maintaining successful payload recovery. Although the proposed implementation also produced visually similar stego images with high SSIM values, image quality was more dependent on the cover image, payload size, and embedding parameters.

A quantitative comparison of detectability was beyond the scope of this project because no automated JPEG steganalysis tool was incorporated into the benchmark. Consequently, neither algorithm was evaluated using statistical steganalysis techniques such as Aletheia. Detection was therefore assessed only through visual quality metrics and successful payload recovery. Future work should include automated steganalysis to provide a more complete comparison of resistance to statistical detection.

The most significant difference between the two algorithms was observed during the robustness evaluation. The proposed implementation was specifically designed to preserve embedded data after JPEG recompression, and this behavior was demonstrated during the benchmark. Using the 75Peacock.jpg image and the HelloWorld.txt payload, the proposed implementation successfully recovered the embedded message after recompression at JPEG quality levels of 90, 75, and 50. Under the same conditions, the F5 implementation recovered the payload only after recompression at quality level 75. Although this represents a single benchmark case rather than a universal result, it demonstrates the compression robustness that the original Wavelet Domain SVD and Adaptive QIM algorithm was designed to achieve.

Another practical difference between the two implementations lies in the embedding process. The proposed implementation performs a wavelet transform, modifies singular values, reconstructs the image through the inverse wavelet transform, and then writes a new JPEG image. As discussed earlier in this report, this additional JPEG encoding step introduces another stage of lossy compression, which likely contributes to the reduced extraction reliability and lower PSNR observed during the benchmark. In contrast, F5 operates directly on the JPEG's quantized DCT coefficients, avoiding the wavelet reconstruction process used by the proposed implementation. While this difference contributes to F5's higher image quality and extraction reliability, the benchmark demonstrates that the proposed implementation provides improved robustness to JPEG recompression in representative test cases.

Overall, the benchmark illustrates the tradeoffs between the two approaches. F5 consistently achieved higher embedding capacity, greater extraction reliability, and higher image quality across the benchmark cases. The proposed Wavelet Domain SVD and Adaptive QIM implementation, however, demonstrated the compression robustness emphasized by the original paper, showing that sacrificing some capacity and extraction reliability can improve resilience to repeated JPEG compression.

Challenges and Future Work

While studying Liu et al.'s original implementation and creating our own, there were a few snags that we hit that blocked us from progressing further and fully accomplishing our development goals.

Incorrect Equation in Source Paper

The biggest challenge was realizing that the paper actually had an extremely unfortunate typo. The equation proposed on page 3 by Liu et al. To calculate the individual block, the Quantization value is incorrect. Simply put, the equation in the paper subtracts from the minimum quantization value which leads to negative quantization values that do not fit in the specified boundaries of 20 being the minimum, and 70 being the maximum. This error in the paper cost vast amounts of time performing different tests and debugging errors in our implementation, which we thought was causing the issues we were having. Ultimately, it was found that the error was in the equation, not our code. Once the error was found, the algorithm's development hit its stride once again. Below is the incorrect equation vs the corrected one:

$$T_i = T_{min} - (T_{max} - T_{min}) \times \frac{\omega_i - \omega_{min}}{\omega_{max} - \omega_{min}}$$

Fig 10: Original Equation from Paper

$$T_i = T_{max} - (T_{max} - T_{min}) \times \frac{\omega_i - \omega_{min}}{\omega_{max} - \omega_{min}}$$

Fig 11: Modified, corrected equation

Loss of a Team Member

As we approached the halfway point of this semester, one team member had dropped the class. This was not communicated to us by the other team members and thus led to a faulty understanding of how work was to be distributed and accomplished. Ultimately, this led to more loads being put on the remaining team members than what was ultimately expected, resulting in compressed timelines and reduced capacity to fully implement our vision for this steganography program implementation.

Future Work

Should more time and resources be available, full implementation of the syndrome-trellis codes (STCs) and embedding cost calculations would have been implemented. Regarding the STCs, no specific framework was planned for implementation, but it can be assumed that Liu et al. based theirs off the one proposed by Fredrich et al. in their paper *Minimizing Additive Distortion in Steganography Using Syndrome-Trellis Codes*. It can be expected that implementing STCs and embedding cost calculations would increase imperceptibility and even reduce the bit error rate since less desirable blocks may be skipped entirely. This stands in line with the goals outlined in the original adaptive QIM algorithm. Both intra-block and inter-block distortion remain a key focal point in the original paper with the explicit goal of lowering detection (Liu et al., 3).

In addition to this, one specific improvement to our algorithm would be to add support for JPEGs of all pixel ratios. Currently, the program only works if the pixel ratios are perfectly divisible by the selected block size (default 4), though this restriction is not necessary. It was implemented in this manner for the sake of simplicity due to the setbacks mentioned above. A future implementation of this program would skip blocks that cross the edge of the image and ensure that no blocks are taken after the end of the image. This would hurt the capacity slightly, but since this is entirely within the user's control, it is a trade-off they should be aware of.

Finally, analysis regarding detectability of the SVD and Adaptive QIM algorithm should be done. Two tools were proposed for future testing, Aletheia and Stegdetect. After upgrading and extending our implementation with the improvements discussed above, utilizing these steganalysis tools on the images produced from our implementation can paint a more coherent and well-rounded picture of the effectiveness of the SVD and Adaptive QIM algorithm.

Conclusion

This project successfully implemented a Python version of the Wavelet Domain SVD and Adaptive QIM JPEG steganography algorithm described by Liu et al. The implementation demonstrated the use of wavelet-domain decomposition, singular value decomposition (SVD), and Adaptive Quantization Index Modulation (Adaptive QIM) to embed hidden information within JPEG images while preserving acceptable visual quality.

Benchmark evaluation showed that the proposed implementation achieved the primary objective of the original paper by demonstrating robustness to JPEG recompression in representative test cases. Although the implementation did not consistently outperform the F5 algorithm in terms of embedding capacity, extraction reliability, or image quality, it successfully illustrated the tradeoff between robustness, imperceptibility, and embedding capacity. The comparison with F5 also highlighted the strengths and limitations of the current implementation while providing a meaningful baseline against an established JPEG steganography technique.

Overall, this project demonstrates that Wavelet Domain SVD and Adaptive QIM is a practical approach for robust JPEG steganography. While additional improvements, including Syndrome-Trellis Codes (STCs), parameter optimization, and more extensive steganalysis testing, would likely improve performance, the implementation successfully reproduced the core concepts of the original algorithm and provided valuable insight into the challenges of designing robust steganographic systems.

Bibliography

J. Liu, X. Song, G. Li and K. Han, "Robust JPEG Image Steganography Using Wavelet Domain SVD and Adaptive QIM," 2023 8th International Conference on Signal and Image Processing (ICSIP), Wuxi, China, 2023, pp. 434-438, doi: 10.1109/ICSIP57908.2023.10270839. https://ieeexplore-ieee-org.libweb.lib.utsa.edu/document/10270839.

A. Westfeld, "F5-A Steganographic Algorithm," vol. 2137, Springer, Berlin, Heidelberg, 2001, pp. 289–302. doi: 10.1007/3-540-45496-9_21. https://scispace.com/papers/f5-a-steganographic-algorithm-4axbc844s4.

T. Filler, J. Judas and J. Fridrich, "Minimizing Additive Distortion in Steganography Using Syndrome-Trellis Codes," in *IEEE Transactions on Information Forensics and Security*, vol. 6, no. 3, pp. 920-935, Sept. 2011, doi: 10.1109/TIFS.2011.2134094. https://ieeexplore.ieee.org/document/5740590.

Resources

jackfengji. F5 Steganography (Python Implementation). https://github.com/jackfengji/f5-steganography

NumPy. https://numpy.org/.

Pillow. https://pillow.readthedocs.io/en/stable/.

PyWavelets. https://pywavelets.readthedocs.io/en/latest/index.html#.